

Weekly Status Report

E-Pharma Management System — Phase 5

Employee	Jainish Nikul Shah	Designation	Software Engineer
Project	Techmentee — E-Pharma Management System	Coordinator	Vaishali Pujari
Reporting Period	Phase 5 — 10 August 2026 – 14 August 2026	Report Date	14 August 2026
Phase	Phase 5 — Performance & Security	Overall Status	Completed (ahead of schedule)

1. Summary

All Phase 5 objectives — PostgreSQL migration, encryption at rest, audit logging, load testing and an accessibility pass — were completed and verified. The database work was implemented as a single dual-backend data layer rather than a one-way migration: the application runs on SQLite with zero configuration for demonstrations and on PostgreSQL for production simply by setting a connection string, with the same automated test suite proving both. Running the suite against a live PostgreSQL server surfaced three real portability defects (foreign-key declaration order, character-encoding of currency and dash characters, and decimal versus floating-point number formatting) that a code review alone would not have caught; all three were fixed and re-verified.

2. Tasks Completed This Phase

Task (Phase 5 scope)	Status	Remarks
PostgreSQL migration	Completed	Dual-backend data layer: SQLite by default, PostgreSQL when DATABASE_URL is set. Placeholders, RETURNING id, schema types and transactions are translated in one place. The complete test suite was executed against a live PostgreSQL server and passes on both backends
Encryption at rest for medical records	Completed	Prescriptions, consultation messages and verification documents are encrypted (Fernet: AES-CBC with HMAC authentication) before storage and decrypted on read, so a stolen database file contains only ciphertext
Audit logging	Completed	New audit_log table records logins including failed attempts, registrations, approvals, orders, prescriptions and content edits; reviewable by administrators through a dedicated, admin-only endpoint
Load testing	Completed	Repeatable benchmark script added (loadtest.sh). Measured locally: approximately 2,100–2,900 requests/second across the hot endpoints, 95th-percentile latency at or below 19 ms, with zero failed requests
Accessibility pass	Completed	Skip-to-content link, visible keyboard focus indicators, ARIA labelling, tab semantics with selection state, live regions for status messages, dialog semantics with focus management and Escape-to-close, alternative text on all images, and reduced-motion support
Testing & documentation	Completed	Test suite grown to 49 assertions — including one that reads the raw database column to prove the stored medical record is ciphertext —

Task (Phase 5 scope)	Status	Remarks
		and verified on both SQLite and PostgreSQL; README and analysis document updated

3. Deliverables

- Dual-backend data layer supporting SQLite and PostgreSQL from one code path
- Encryption module protecting prescriptions, consultation messages and verification documents at rest
- Audit trail with an administrator review endpoint
- Repeatable load-test script with recorded baseline measurements
- Accessibility improvements across the entire interface
- Test suite expanded to 49 assertions, passing on both database backends; documentation updated

4. Measured Results

- Throughput: approximately 2,100–2,900 requests/second on the read endpoints; 95th-percentile latency at or below 19 ms; zero failed requests
- Correctness: 49 of 49 automated assertions pass on SQLite and on PostgreSQL
- Security: the raw database column for a stored prescription was confirmed to contain ciphertext, while the API returns the correct plaintext to the authorised user

5. Plan for Next Phase (Phase 6 — Deployment & Handover)

- Production deployment on the agreed hosting environment, served by a production application server behind TLS
- Provision the managed PostgreSQL instance and load the encryption key from a secrets manager
- Final documentation, demonstration and handover

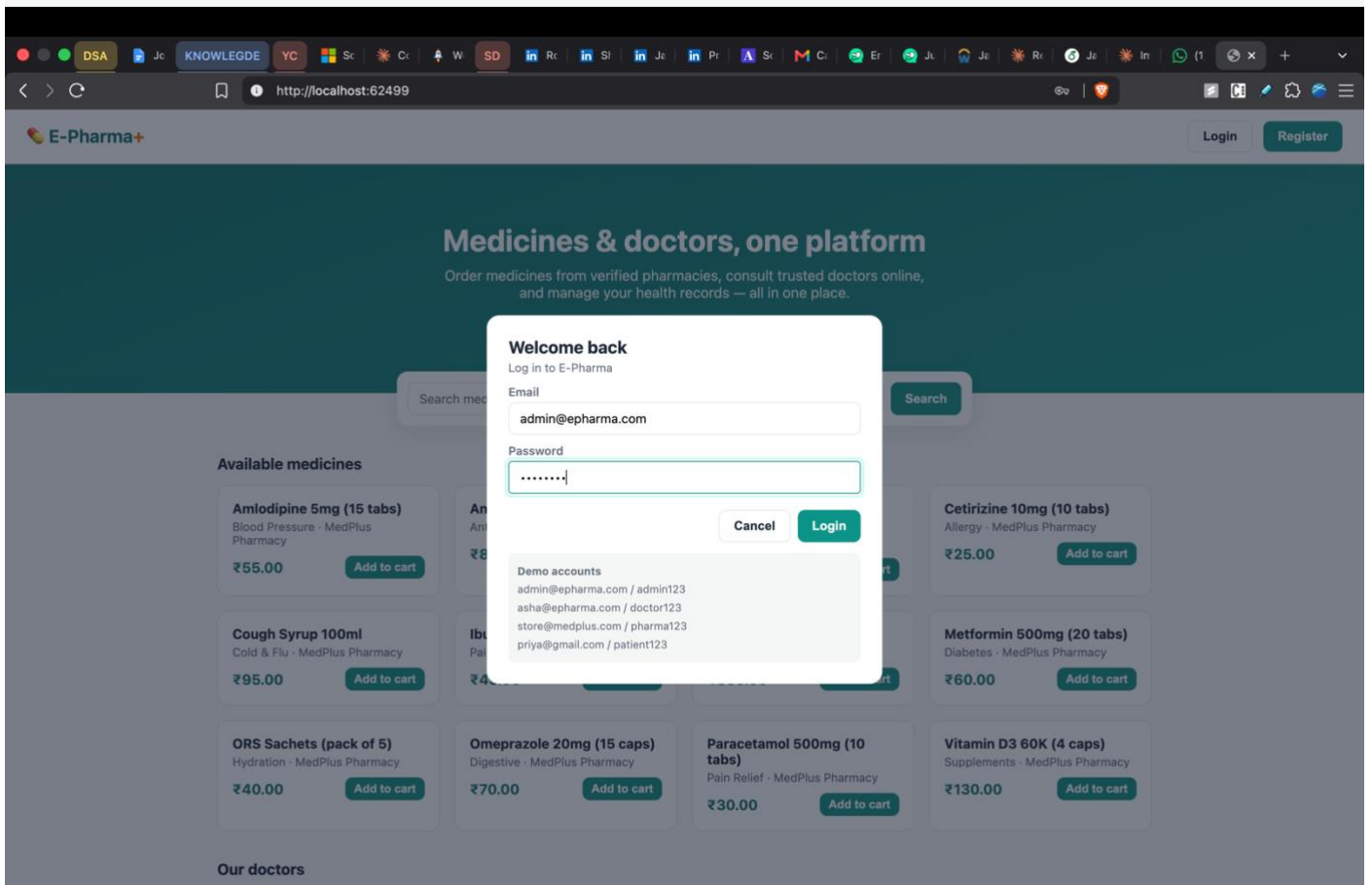
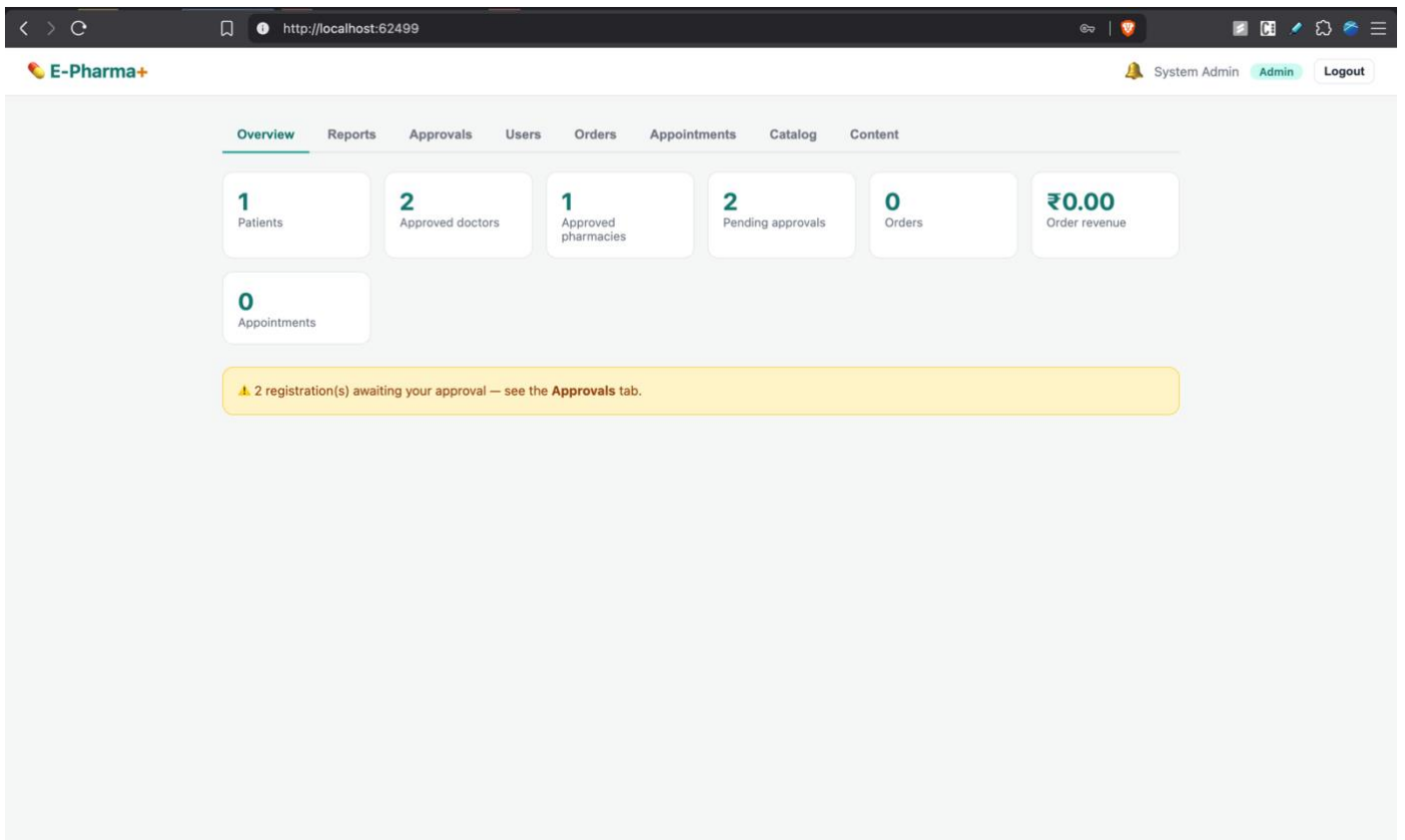
6. Issues / Blockers

None blocking. Before deployment, three confirmations are requested from the coordinator: the target hosting environment and managed PostgreSQL instance; the payment provider to license for go-live (Stripe or Razorpay, both integrated); and the SMS/email provider for live notification delivery. A data-retention policy and consent capture remain outstanding for full HIPAA/GDPR alignment and are proposed for Phase 6.

Prepared by: Jainish Nikul Shah, Software Engineer

Submitted to: Vaishali Pujari, Project Coordinator

Few Screenshot as the reference



Overview Reports Approvals Users Orders Appointments Catalog Content

ROLE	NAME	DETAILS	ACTIONS
Pharmacy	Sunita Rao store@healthkart.com	HealthKart Chemist · License DL-MH-67890 · GSTIN 27FGHIJ5678K2Z9	View documents Approve Reject
Doctor	Dr. Kunal Shah kunal@epharm.com	MBBS, MS (Ortho) · Orthopedics · fee ₹700.00	View documents Approve Reject

Below code shows the implementation for the Admin Dashboard Backend APIs

```
# ===== admin =====

@api
def admin_users(request):
    """GET /api/admin/users?status=&role= — admin. All non-admin users, optionally filtered."""
    user, err = require_auth(request, "admin")
    if err:
        return err
    sql, params = "SELECT * FROM users WHERE role != 'admin'", []
    if request.GET.get("status"):
        sql += " AND status = ?"
        params.append(request.GET["status"])
    if request.GET.get("role"):
        sql += " AND role = ?"
        params.append(request.GET["role"])
    return J([db.public_user(u) for u in db.query(sql + " ORDER BY id DESC", params)])

@api
def admin_user_detail(request, id):
    """PATCH /api/admin/users/<id> — admin approves or rejects a pending doctor/pharmacy."""
    user, err = require_auth(request, "admin")
    if err:
        return err
    target = db.get("SELECT * FROM users WHERE id = ?", (id,))
    if not target:
```

```

    return J({"error": "User not found"}, 404)

status = request.data.get("status")

if status not in ("approved", "rejected"):
    return J({"error": "Invalid status"}, 400)

db.run("UPDATE users SET status = ? WHERE id = ?", (status, target["id"]))
db.audit(user, f"user.{status}", f"{target['role']} #{target['id']} ({target['email']})")
notify(target["id"], f"Your {target['role']} account was {status} by the admin")
return J(db.public_user(db.get("SELECT * FROM users WHERE id = ?", (target["id"],))))

```

@api

```

def admin_stats(request):
    """GET /api/admin/stats — admin. Headline KPI counts for the dashboard overview."""
    user, err = require_auth(request, "admin")
    if err:
        return err

    def c(sql):
        return db.get(sql)["c"]

    return J({
        "patients": c("SELECT COUNT(*) AS c FROM users WHERE role = 'patient'"),
        "doctors": c("SELECT COUNT(*) AS c FROM users WHERE role = 'doctor' AND status = 'approved'"),
        "pharmacies": c("SELECT COUNT(*) AS c FROM users WHERE role = 'pharmacy' AND status = 'approved'"),
        "pending_approvals": c("SELECT COUNT(*) AS c FROM users WHERE status = 'pending'"),
        "orders": c("SELECT COUNT(*) AS c FROM orders"),
        "revenue": db.get("SELECT COALESCE(SUM(total), 0) AS s FROM orders")["s"],
        "appointments": c("SELECT COUNT(*) AS c FROM appointments"),
    })

```

@api

```

def admin_reports(request):
    """GET /api/admin/reports — admin. Aggregation queries powering the Reports charts."""
    user, err = require_auth(request, "admin")
    if err:
        return err

    # NOTE: CAST(...AS TEXT) and CAST(...AS NUMERIC) keep these portable across SQLite and Postgres
    # (Postgres can't substr a timestamp, and its ROUND needs numeric input).

    return J({
        "revenue_by_day": db.query("SELECT substr(CAST(created_at AS TEXT),1,10) AS day, ROUND(CAST(SUM(total) AS NUMERIC),2) AS revenue, COUNT(*) AS orders FROM orders GROUP BY substr(CAST(created_at AS TEXT),1,10) ORDER BY day"),
        "orders_by_status": db.query("SELECT status, COUNT(*) AS count FROM orders GROUP BY status"),
    })

```

```

    "top_medicines": db.query("SELECT name, SUM(qty) AS units, ROUND(CAST(SUM(price*qty) AS NUMERIC),2) AS revenue
FROM order_items GROUP BY name ORDER BY units DESC LIMIT 5"),

    "revenue_by_pharmacy": db.query("SELECT u.store_name AS pharmacy, ROUND(CAST(SUM(o.total) AS NUMERIC),2) AS
revenue, COUNT(*) AS orders FROM orders o JOIN users u ON u.id=o.pharmacy_id GROUP BY u.store_name ORDER BY
revenue DESC"),

    "appointments_by_status": db.query("SELECT status, COUNT(*) AS count FROM appointments GROUP BY status"),

    "consultations": db.get("SELECT COUNT(*) AS total, COALESCE(SUM(d.fee),0) AS revenue FROM appointments a JOIN
users d ON d.id=a.doctor_id WHERE a.status='completed'"),

    })

```

@api

def admin_taxonomy(request):

"""POST /api/admin/taxonomy — admin adds a category or specialty (ignored if it already exists)."""

user, err = require_auth(request, "admin")

if err:

return err

b = request.data

if b.get("type") not in ("category", "specialty") or not non_empty(b.get("name")):

return J({"error": "A type (category/specialty) and name are required"}, 400)

db.run("INSERT INTO taxonomy (type, name) VALUES (?, ?) ON CONFLICT DO NOTHING", (b["type"], b["name"].strip()))

return J(db.get("SELECT * FROM taxonomy WHERE type = ? AND name = ?", (b["type"], b["name"].strip())))

@api

def admin_taxonomy_detail(request, id):

"""DELETE /api/admin/taxonomy/<id> — admin removes a category or specialty."""

user, err = require_auth(request, "admin")

if err:

return err

db.run("DELETE FROM taxonomy WHERE id = ?", (id,))

return J({"ok": True})

@api

def admin_cms(request, slug):

"""PUT /api/admin/cms/<slug> — admin creates or updates a CMS page (upsert on slug)."""

user, err = require_auth(request, "admin")

if err:

return err

b = request.data

if not non_empty(b.get("title")) or not non_empty(b.get("body")):

return J({"error": "Title and body are required"}, 400)

db.run("INSERT INTO cms_pages (slug, title, body, updated_at) VALUES (?, ?, ?, CURRENT_TIMESTAMP) "

"ON CONFLICT(slug) DO UPDATE SET title = excluded.title, body = excluded.body, updated_at = excluded.updated_at",

```
        (slug, b["title"].strip(), b["body"]))
db.audit(user, "cms.update", slug)
return J(db.get("SELECT * FROM cms_pages WHERE slug = ?", (slug,)))

@api
def admin_audit(request):
    """GET /api/admin/audit — admin. The most recent 100 audit-trail entries (security review)."""
    user, err = require_auth(request, "admin")
    if err:
        return err
    return J(db.query("SELECT * FROM audit_log ORDER BY id DESC LIMIT 100"))
```